

Artificial Intelligence

Evaluating AI agents: Real-world lessons from building agentic systems at Amazon

by Yunfei Bai, Allie Colin, Kashif Imran, and Winnie Xiong | on 18 FEB 2026 | in [Amazon Bedrock AgentCore](#), [Artificial Intelligence](#), [Best Practices](#), [Generative AI](#) | [Permalink](#) | [Comments](#) | [Share](#)

The generative AI industry has undergone a significant transformation from using [large language model](#) (LLM)-driven applications to [agentic AI systems](#), marking a fundamental shift in how AI capabilities are architected and deployed. While early [generative AI](#) applications primarily relied on LLMs to directly generate text and respond to prompts, the industry has evolved from those static, prompt-response paradigms toward autonomous agent frameworks to build dynamic, goal-oriented systems capable of tool orchestration, iterative problem-solving, and adaptive task execution in production environments.

We have witnessed this evolution in Amazon; since 2025, there have been thousands of agents built across Amazon organizations. While single-model benchmarks serve as a crucial foundation for assessing individual LLM performance in LLM-driven applications, agentic AI systems require a fundamental shift in evaluation methodologies. The new paradigm assesses not only the underlying model performance but also the emergent behaviors of the complete system, including the accuracy of tool selection decisions, the coherence of multi-step reasoning processes, the efficiency of memory retrieval operations, and the overall success rates of task completion across production environments.

In this post, we present a comprehensive evaluation framework for Amazon agentic AI systems that addresses the complexity of agentic AI applications at Amazon through two core components: a generic evaluation workflow that standardizes assessment procedures across diverse agent implementations, and an agent evaluation library that provides systematic measurements and metrics in [Amazon Bedrock AgentCore Evaluations](#), along with Amazon use case-specific evaluation approaches and metrics. We also share best practices and experiences captured during engagements with multiple Amazon teams, providing actionable insights for AWS developer communities facing similar challenges in evaluating and deploying agentic AI systems within their own business contexts.

AI agent evaluation framework in Amazon

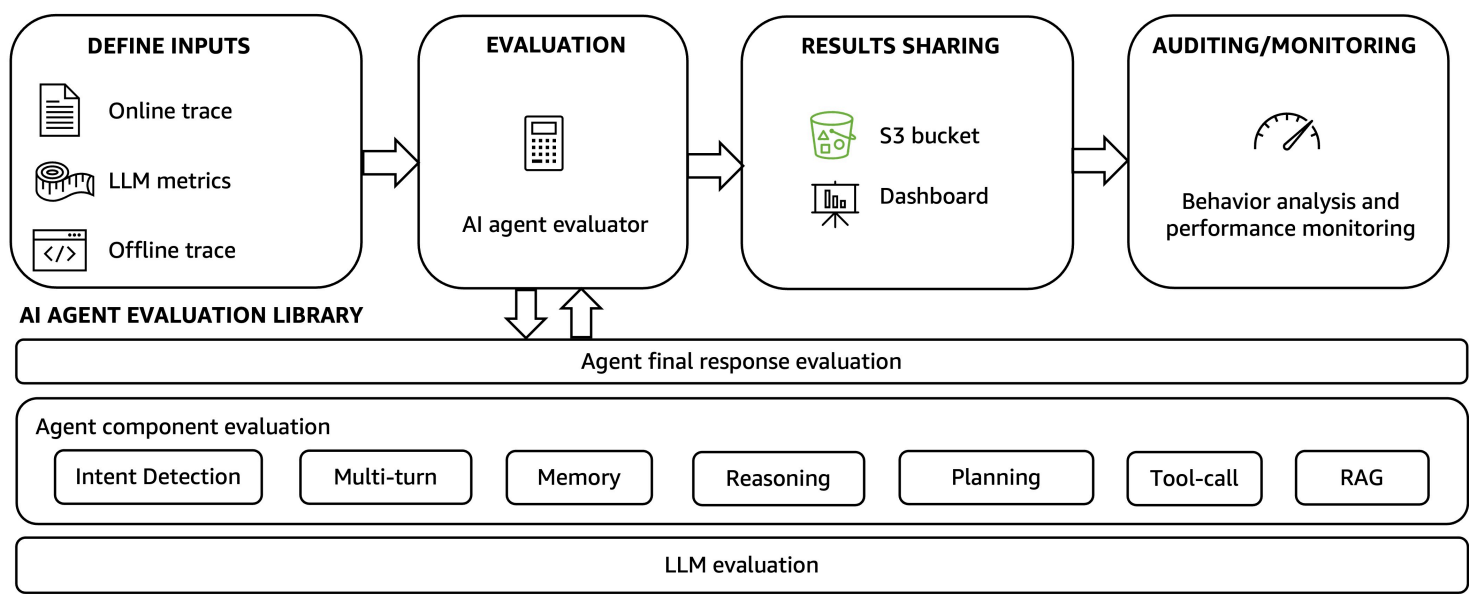
When builders design, develop, and evaluate AI agents, they face significant challenges. Unlike traditional LLM-driven applications that only generate responses to isolated prompts, AI agents autonomously pursue goals through multi-step reasoning, tool use, and adaptive decision-making across multi-turn interactions. Traditional LLM evaluation methods treat agent systems as black boxes and evaluate only the final outcome, failing to provide sufficient insights to determine why AI agents fail or pinpoint the root causes. Although multiple specific evaluation tools are available in the industry, builders must navigate among them and consolidate results with significant manual efforts. Additionally, while agent development frameworks, such as [Strands Agents](#), [LangChain](#), and [LangGraph](#), have built-in evaluation modules, builders want a framework-agnostic evaluation approach rather than being locked into methods within a single framework.

Additionally, robust self-reflection and error handling in AI agents requires systematic assessment of how agents detect, classify, and recover from failures across the execution lifecycle in reasoning, tool-use, memory handling, and

action taking. For example, the evaluation frameworks must measure the agent’s ability to recognize diverse failure scenarios such as inappropriate planning from the reasoning model, invalid tool invocations, malformed parameters, unexpected tool response formats, authentication failures, and memory retrieval errors. A production-grade agent must demonstrate consistent error recovery patterns and resilience in maintaining the coherence of user interactions after encountering exceptions.

To meet these needs, AI agents deployed in production environments at scale require continuous monitoring and systematic evaluation to promptly detect and mitigate agent decay and performance degradation. This demands that the agent evaluation framework streamline the end-to-end process and provide near real-time issue detection, notification, and problem resolution. Finally, incorporating [human-in-the-loop](#) (HITL) processes is essential to audit evaluation results, helping to ensure the reliability of system outputs.

To address these challenges, we propose a holistic agentic AI evaluation framework, as shown in the following figure. The framework contains two key components: an automated AI agent evaluation workflow and an AI agent evaluation library.



The automated AI agent evaluation workflow drives the holistic evaluation approach with four steps.

Step 1: Users define inputs for evaluation, typically trace files from agent execution. These can be offline traces collected after the agent completes the task and uploaded to the framework using a unified API access point or online traces where users can define evaluation dimensions and metrics.

Step 2: The AI agent evaluation library is used to automatically generate default and user-defined evaluation metrics. The methods in the library are described in the next list.

Step 3: The evaluation results are shared through an [Amazon Simple Storage Service \(Amazon S3\)](#) bucket or a dashboard that visualizes the agent trace observability and evaluation results.

Step 4: Results are analyzed through agent performance auditing and monitoring. Builders can define their own rules to send notifications upon agent performance degradation and can take action to resolve problems. Builders

can also HITL mechanisms to schedule periodic human audits of agent trace subsets and evaluation results, improving consistent agent quality and performance.

The AI agent evaluation library operates across three layers: calculating and generating evaluation metrics for the agent's final output, assessing individual agent components, and measuring the performance of the underlying LLMs that power the agent.

- **Bottom layer:** Benchmarks multiple foundation models to select the appropriate models powering the AI agent and determine how different models impact the agent overall quality and latency.
- **Middle layer:** Evaluates the performance of the components of the agent, including intent detection, multi-turn conversation, memory, LLM reasoning and planning, tool-use, and others. For example, the middle layer determines whether the agent understands user intents correctly, how the LLM drives agentic workflow planning through [chain-of-thought](#) (CoT) reasoning, whether the tool selection and execution are aligned with the agentic plan, and if the plan is completed successfully.
- **Upper layer:** Assesses the agent's final response, the task completion, and whether the agent meets the goal defined in the use case. It also covers overall responsibility and safety, the costs, and the customer experience impacts.

[Amazon Bedrock AgentCore Evaluations](#) provides automated assessment tools to measure how well your agent or tools perform specific tasks, handle edge cases, and maintain consistency across different inputs and contexts. In the agent evaluation library, we provide a set of pre-defined evaluation metrics for the agent's final response and its components, based on the built-in configurations, evaluators, and metrics of AgentCore Evaluations. We further extended the evaluation library with specialized metrics designed for the heterogeneous scenario complexity and application-specific requirements of Amazon. The primary metrics in the library include

- **Final response quality:**
 - **Correctness:** The factual accuracy and correctness of an AI assistant's response to a given task.
 - **Faithfulness:** Whether an AI assistant's response remains consistent with the conversation history.
 - **Helpfulness:** How effectively an AI assistant's response helps users appropriately address query and progress toward their goals.
 - **Response relevance:** How well an AI assistant's response addresses the specific question or request.
 - **Conciseness:** How efficiently an AI assistant communicates information, for instance, whether the response is appropriately brief without missing key information.
- **Task completion:**
 - **Goal success:** Did the AI assistant successfully complete all user goals within a conversation session.
 - **Goal accuracy:** Compares the output to the ground truth.
- **Tool use:**
 - **Tool selection accuracy:** Did the AI assistant choose the appropriate tool for a given situation.
 - **Tool parameter accuracy:** Did the AI assistant correctly use contextual information when making tool calls.
 - **Tool call error rate:** The frequency of failures when an AI assistant makes tool calls.

- **Multi-turn function calling accuracy:** Are multiple tools being called and how often the tools are called in the correct sequence.
- **Memory:**
 - **Context retrieval:** Assesses the accuracy of findings and surfaces the most relevant contexts for a given query from memory, prioritizing relevant information based on similarity or ranking, and balancing precision and recall.
- **Multi-turn:**
 - **Topic adherence classification:** If a multi-turn conversation includes multiple topics, assesses whether the conversation stays on predefined domains and topics during the interaction.
 - **Topic adherence refusal:** Determines if the AI agent refuse to answer questions about a topic.
- **Reasoning:**
 - **Grounding accuracy:** Does the model understand the task, appropriately select tools, and is the CoT aligned with the provided context and data returned by external tools.
 - **Faithfulness score:** Measures logical consistency across the reasoning process.
 - **Context score:** Is each step taken by the agent contextually grounded.
- **Responsibility and safety:**
 - **Hallucination:** Do the outputs align with established knowledge, verifiable data, logical inference, or include any elements that are implausible, misleading, or entirely fictional.
 - **Toxicity:** Do the outputs contain language, suggestions, or attitudes that are harmful, offensive, disrespectful, or promote negativity. This include content that might be aggressive, demeaning, bigoted, or excessively critical without constructive purpose.
 - **Harmfulness:** Is there potentially harmful content in an AI assistant's response, including insults, hate speech, violence, inappropriate sexual content, and stereotyping.

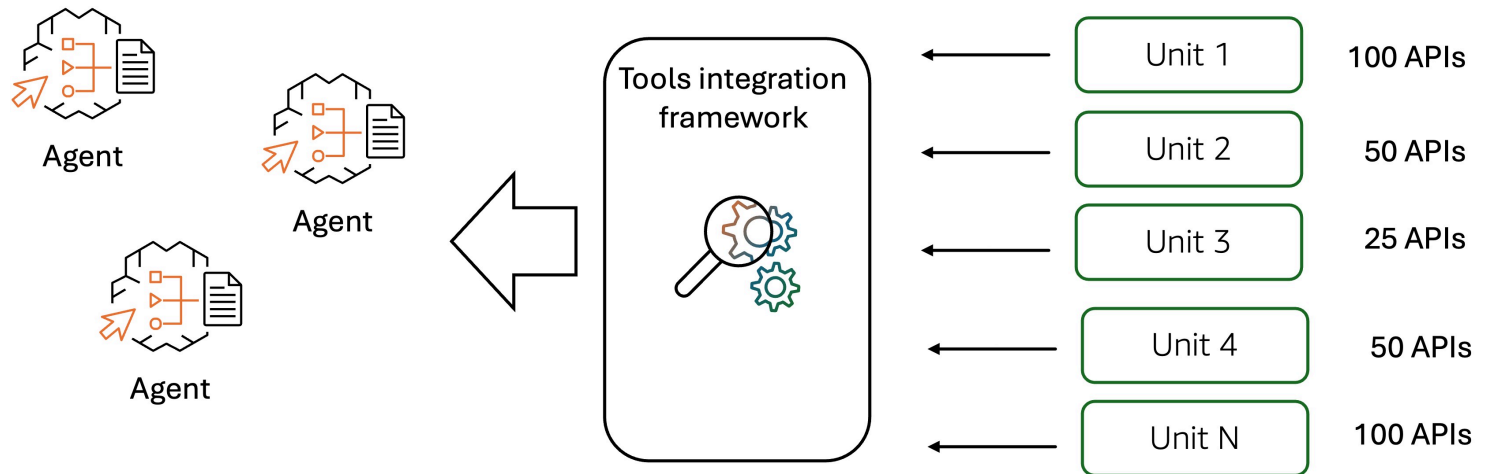
See [AgentCore evaluation templates](#) for other agent output quality metrics, or how to create custom evaluators that are tailored to your specific use cases and evaluation requirements.

Evaluating real-world agent systems used by Amazon

In the past few years, Amazon has been working to advance its approach in building agentic AI applications to address complex business challenges, streamlining business processes, improving operational efficiency, and optimizing business outcomes—moving from early experimentation to production-scale deployments across multiple business units. These agentic AI applications operate at enterprise scale and are deployed across AWS infrastructure, transforming how work gets done across global operations within Amazon. In this section, we introduce a few real-world agentic AI use cases from Amazon, to demonstrate how Amazon teams improve AI agent performance through holistic evaluation using the framework discussed in the previous section.

Evaluating tool-use in the Amazon shopping assistant AI agent

To deliver a smooth shopping experience to Amazon consumers, the Amazon shopping assistant can seamlessly interact with numerous APIs and web services from underlying Amazon systems, as shown in the following figure. The AI agent needs to onboard hundreds, sometimes thousands, of tools from underlying Amazon systems to engage in long-running multi-turn conversations with the consumer. The agent uses these tools to deliver a personalized experience that includes customer profiling, product and inventory discovery, and order placement. However, manually onboarding so many enterprise APIs and web services to an AI agent is a cumbersome process that typically takes months to complete.

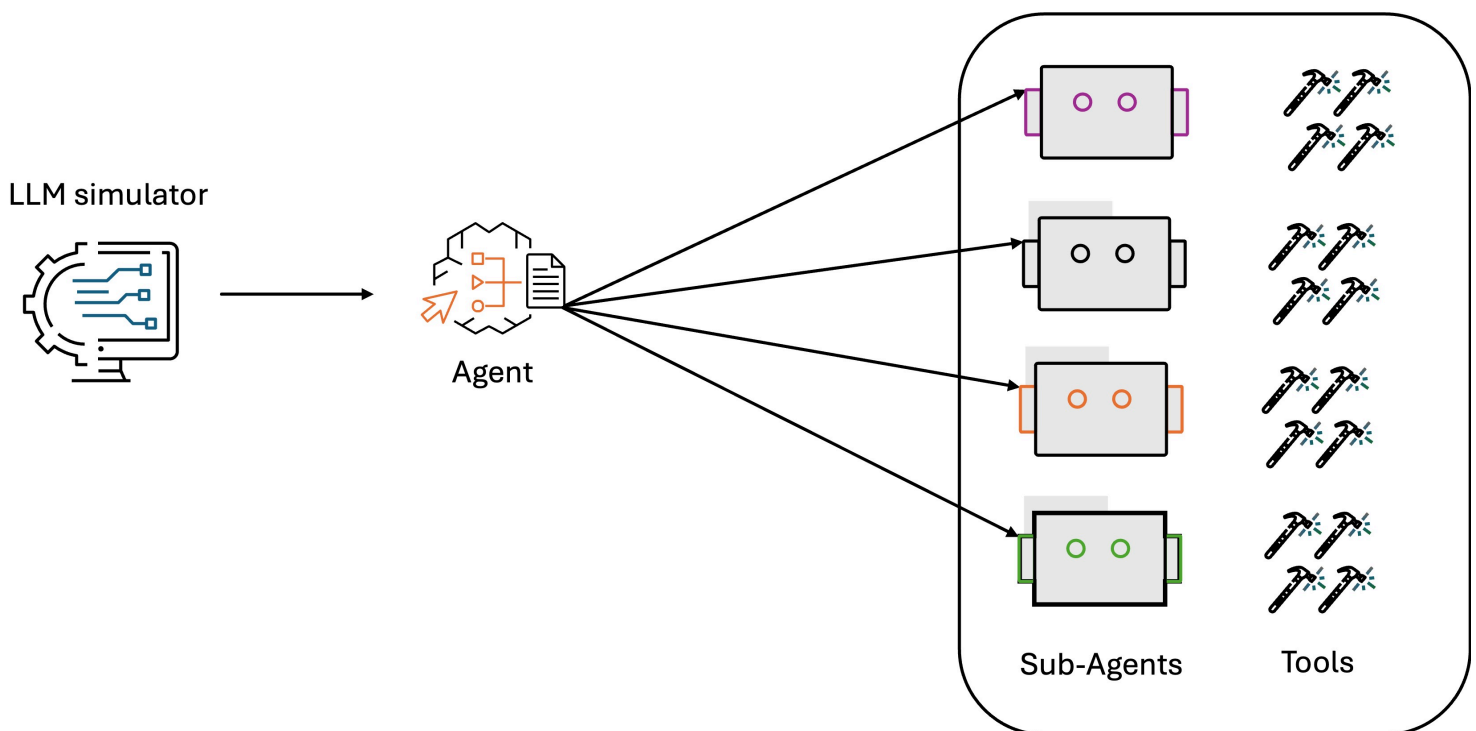


Transforming legacy APIs and web services into agent-compatible tools requires the systematic definition of structured schemas and semantic descriptions for the endpoints of the API and web services, enabling the agent's reasoning and planning mechanisms to accurately identify and select contextually appropriate tools during task execution. Poorly defined tool schemas and imprecise semantic descriptions result in erroneous tool selection during agent runtime, leading to the invocation of irrelevant APIs that unnecessarily expand the context window, increase inference latency, and escalate computational costs through redundant LLM calls. To address these challenges, Amazon defined cross-organizational standards for tool schema and description formalization, creating a governance framework that specifies mandatory compliance requirements for all builder teams involved in tool development and agent integration. This standardization initiative establishes uniform specifications for tool interfaces, parameter definitions, capability descriptions, and usage constraints, helping to ensure that tools developed across diverse organizational units maintain consistent structural patterns and semantic clarity to produce reliable agent-tool interactions. All builder teams engaged in tool development and agent integration must conform to these architectural specifications, which prescribe standardized formats for tool signatures, input validation schemas, output contracts, and human-readable documentation. This helps ensure consistency in tool representation across the enterprise agentic systems. Furthermore, manually defining tool schemas and descriptions for hundreds or thousands of tools represents a significant engineering burden, and the complexity escalates substantially when multiple APIs require coordinated orchestration to accomplish composite tasks. Amazon builders implemented an API self-onboarding system that uses LLMs to automate the generation of standardized tool schemas and descriptions. This significantly improved the efficiency in onboarding large numbers of APIs and services into agent-compatible tools, accelerating integration timelines and reducing manual engineering overhead. To evaluate the tool-selection and tool-use after integration of the APIs is completed, Amazon teams created golden datasets for regression testing. The datasets are generated synthetically using LLMs from historical API invocation logs upon user queries. Using pre-defined tool-selection and tool-use metrics such as tool selection accuracy, tool parameter accuracy, and multi-turn function call accuracy, the Amazon builders can systematically evaluate the shopping

assistant AI agent's capability to correctly identify appropriate tools, populate their parameters with accurate values, and maintain coherent tool invocation sequences across conversational turns. As the agent continues to evolve, the ability to rapidly and reliably integrate new APIs as tools in the agent and evaluate the tool-use performance becomes increasingly crucial. The objective assessment of agent's functional reliability in production environments effectively reduces development overhead while maintaining robust performance in the agentic AI applications.

Evaluating user intent detection in the Amazon customer service AI agent

In the Amazon customer-service landscape, AI agents are instrumental in handling customer inquiries and resolving issues. At the heart of these systems lies a crucial capability: an orchestration AI agent using its reasoning model to accurately detect customer intent, which determines whether a customer's query is correctly understood and routed to the appropriate specialized resolver implemented by agent tools or subagents, as shown in the following figure. The stakes are high when it comes to intent detection accuracy. When the customer service agent misinterprets a customer's intent, it can trigger a cascade of problems: queries get routed to the wrong specialized resolvers, customers receive irrelevant responses, and frustration builds. This impacts customer experience and leads to increased operational costs as more customers seek intervention from human agents.

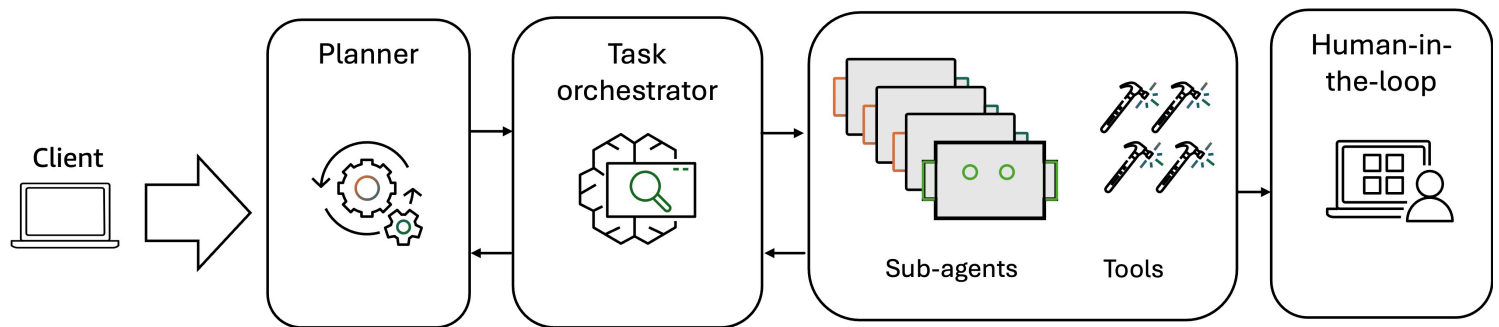


To evaluate the agent's reasoning capability for intent detection, the Amazon team developed an LLM simulator that uses LLM driven virtual customer personas to simulate diverse user scenarios and interactions. The evaluation is mainly focused on correctness of the intent generated by the orchestration agent and routing to the correct subagent. The simulation dataset contains a set of user query and ground truth intent pairs collected from anonymized historical customer interactions. Using the simulator, the orchestration agent generates the intents upon the user queries in the simulation dataset. By comparing the agent response intent to the ground truth intent, we can validate if the agent-generated intents comply with the ground truth.

In addition to the intent correctness, the evaluation covers the task completion—the agent’s final response and intent resolution—as the final goal of the customer service tasks. For the multi-turn conversation, we also include the metrics of topic adherence classification and topic adherence refusal to help ensure conversational coherence and user experience quality. As AI customer service systems continue to evolve, the importance of robust agent reasoning evaluation for user intent detection only grows, the impact extends beyond immediate customer satisfaction. It also optimizes customer service operation efficiency and service delivery costs, and so maximizes the return on AI investments.

Evaluating multi-agent systems at Amazon

As enterprises increasingly confront multifaceted challenges in complex business environments, ranging from cross-functional workflow orchestration to real-time decision-making under uncertainty, Amazon teams are progressively adopting multi-agent system architectures that decompose monolithic AI solutions into specialized, collaborative agents capable of distributed reasoning, dynamic task allocation, and adaptive problem-solving at scale. One example is the Amazon seller assistant AI agent that encompasses collaborations among multiple AI agents, depicted in the following flow chart.



The agentic workflow, beginning with an LLM planner and task orchestrator, receives user requests, decomposes complex tasks into specialized subtasks, and intelligently assigns each subtask to the most appropriate underlying agent based on their capabilities and current workload. The underlying agents then operate autonomously, executing their assigned tasks by using their specialized tools, reasoning capabilities, and domain expertise to complete objectives without requiring continuous oversight from the orchestrator. Upon task completion the specialized agents communicate back to the orchestration agent, reporting task status updates, completion confirmations, intermediate results, or escalation requests when they encounter scenarios beyond their operational boundaries. The orchestration agent aggregates these responses, monitors overall progress, handles dependencies between subtasks, and synthesizes the collective outputs into a coherent final result that addresses the original user request. To evaluate this multi-agent collaboration process, the evaluation workflow accounts for both individual agent performance and the overall collective system dynamics. In addition to evaluating the overall task execution quality and performance of specialized agents in task completion, reasoning, tool-use and memory retrieval, we also need to measure the interagent communication patterns, coordination efficiency, and task handoff accuracy. For this, Amazon teams use the metrics such as the planning score (successful subtask assignment to subagents), communication score (interagent communication messages for subtask completion), and collaboration success rate (percentage of successful sub-task completion). In multi-agent systems evaluation, HITL becomes critical because of the increased complexity and potential for unexpected emergent behaviors that automated metrics might fail to capture. Human intervention in the evaluation workflow provides essential oversight for assessing inter-

agent communication to identify coordination failure in specific edge cases, evaluating the appropriateness of agent specialization and whether task decomposition aligns with agent capabilities, and validating potential conflict resolution strategies when agents produce contradictory recommendations. It also helps ensure logical consistency when multiple agents contribute to a single decision, and that the collective agent behavior serves the intended business objective. These are the dimensions that are difficult to quantify through automated metrics alone but are critical for production deployment success.

Lessons learned and best practices

Through extensive engagements with Amazon product and engineering teams deploying agentic AI systems in production environments, we have identified critical lessons learned and established best practices that address the unique challenges of evaluating autonomous agent architectures at scale.

- **Holistic evaluation across multiple dimensions:** Agentic application evaluation must extend beyond traditional accuracy metrics to encompass a comprehensive assessment framework that covers agent quality, performance, responsibility, and cost. Quality evaluation includes measuring reasoning coherence, tool selection accuracy, and task completion success rates across diverse scenarios. Performance assessment captures latency, throughput, and resource utilization under production workloads. Responsibility evaluation addresses safety, toxicity, bias mitigation, hallucination detection, and guardrails to align with organizational policies and regulatory requirements. Cost analysis quantifies both direct expenses including model inference, tool invocation, data processing, and indirect costs such as human efforts and error remediation. This multi-dimensional approach helps ensure holistic optimization across balanced trade-offs.
- **Use case and application-specific evaluation:** Besides the standardized metrics discussed in the previous sections, application-specific evaluation metrics also contribute to the overall application assessment. For instance, customer service applications require metrics such as customer satisfaction scores, first-contact resolution rates, and sentiment analysis scores to measure final business outcomes. This approach requires close collaboration with domain experts to define meaningful success criteria, define appropriate metrics, and create evaluation datasets that reflect real-world operational complexity to complete the assessment process.
- **Human-in-the-loop (HITL) as a critical evaluation component:** As discussed in the multi-agent system evaluation case, HITL is indispensable, particularly for high-stakes decision scenarios. It provides essential evaluation of agent reasoning chains, the coherence of multi-step workflows, and the alignment of agent behavior with business requirements. HITL also helps provide ground truth labels for building golden testing datasets, and calibration of LLM-as-a-judge in the automatic evaluator to align with human preferences.
- **Continuous evaluation in production environments:** It's essential to maintain quality because the pre-deployment evaluation might not fully capture the performance characteristics. Also, production evaluation monitors real-world performance across diverse user behaviors, usage patterns, and edge cases not represented before production deployment to identify performance degradation over time. You can track key metrics through operational dashboards, implement alert thresholds, automate anomaly detection process, and establish feedback loops. When the issues are detected, you can start model retraining, refine context engineering, and align with your ultimate business objectives.

Conclusion

As AI systems become increasingly complex, the importance of a thorough AI agent evaluation approach cannot be overstated. Through holistic evaluation across quality, performance, responsibility, and cost dimensions, in addition to continuous production monitoring and human-in-the-loop validation, the full lifecycle of agentic AI deployment from development to production can be addressed. You can learn from the presented examples, best practices, and lessons learned in this post—many of which are available in Amazon Bedrock AgentCore Evaluations—to accelerate your own agentic AI initiatives while avoiding common pitfalls in evaluation design and implementation.

About the authors



Yunfei Bai

Yunfei Bai is a Principal Applied AI Architect at AWS. With a background in AI/ML, data science, and analytics, Yunfei helps customers adopt AWS services to deliver business results. He designs AI/ML and data analytics solutions that overcome complex technical challenges and drive strategic objectives. Yunfei has a PhD in Electronic and Electrical Engineering. Outside of work, Yunfei enjoys reading and music.



Winnie Xiong

Winnie Xiong is a Senior Technical Product Manager on the Amazon's Benchmarking team. She partners with engineers and scientists to build AI and data solutions that solve complex business challenges for Amazon teams. Her expertise span across model evaluation, agent evaluation, and data management.



Allie Colin

Allie Colin is a Head of Product and Science at Amazon's Benchmarking team. She leads a team of scientists and product managers building tools that help Amazonians test their products for quality through the lens of real customer experiences. Previously, she worked at MicroStrategy as Chief of Staff to the CTO, as well as at Deutsche Bank and Northwestern Mutual. Outside of work, Allie is a mom of four who loves the nightly comedy show they put on and enjoys anything that gets her outdoors—hiking, swimming, and traveling.



Kashif Imran

Kashif Imran is a seasoned engineering and product leader with deep expertise in AI/ML, cloud architecture, and large-scale distributed systems. With a decade of experience at AWS, Kashif has

5/31/26, 2:38 PM

Evaluating AI agents: Real-world lessons from building agentic systems at Amazon | Artificial Intelligence

driven innovation across cloud and AI technologies. Currently a Senior Manager at Amazon Prime Video, he leads AI-native engineering teams building scalable agentic AI solutions to drive business transformation.

 Like (1)

 Share

2 Comments

Log in to comment

Sort by: **Top** ▾